

# Terminal 3 ADK — Bounty Submission

**Repo:** <https://github.com/G-ojies/terminal3-adk> **Live contract:**

z:f39cbc5ab038a4b3fa862e025f971485690864e4:eligibility @ 0.1.5 (contract id 717) **Live verifier:**

<https://verifier-jade.vercel.app/api/eligibility> **DID:**

did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4 **Cluster:** cn-api.sg.testnet.t3n.terminal3.io

**Screenshots:** <https://github.com/G-ojies/terminal3-adk/tree/main/screenshots>

---

## Screenshots

All six are captures of real runs against the live testnet, with the command printed at the top of each. Reproducible via `scripts/shoot.sh`.

### Screenshot 1 — Registering the Rust TEE contract on testnet

```
$ CONTRACT_VERSION=0.1.5 npx tsx --env-file=.env scripts/deploy.ts eligibility

tenant: did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4
status: active label: testnet-dev
quotas: max_contracts=10 max_maps=50 max_wasm_bytes=1048576

registering z:f39cbc5ab038a4b3fa862e025f971485690864e4:eligibility (184 KiB)
registered as contract id 717

z:f39cbc5ab038a4b3fa862e025f971485690864e4:secrets: already exists, continuing
map z:f39cbc5ab038a4b3fa862e025f971485690864e4:secrets -> readers/writers = [717]
z:f39cbc5ab038a4b3fa862e025f971485690864e4:policies: already exists, continuing
map z:f39cbc5ab038a4b3fa862e025f971485690864e4:policies -> readers/writers = [717]
z:f39cbc5ab038a4b3fa862e025f971485690864e4:attestations: already exists, continuing
map z:f39cbc5ab038a4b3fa862e025f971485690864e4:attestations -> readers/writers = [717]

sealed 'verifier_api_key' into z:f39cbc5ab038a4b3fa862e025f971485690864e4:secrets
seeded policy 'adult-eu' (verifier: https://verifier-jade.vercel.app/api/eligibility)

done. script_name = z:f39cbc5ab038a4b3fa862e025f971485690864e4:eligibility @ 0.1.5, contract_id = 717

[exit 0]
```

### Screenshot 2 — The full flow: issue, reuse, verify, reject tampered

```
$ npx tsx --env-file=.env scripts/seed-policy.ts demo-adult-2 NG GB US DE && POLICY_ID=demo-adult-2 npx tsx --env-file=.env scripts/invoke.ts

seeded policy 'demo-adult-2': min_age=18 allowed=[NG, GB, US, DE]
did: did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4
contract: z:f39cbc5ab038a4b3fa862e025f971485690864e4:eligibility @ 0.1.5
verifier: verifier-jade.vercel.app

granted did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4
-> z:f39cbc5ab038a4b3fa862e025f971485690864e4:eligibility (hosts: verifier-jade.vercel.app)

check-eligibility (first call)
-> {"policy_id":"demo-adult-2","subject":"f39cbc5ab038a4b3fa862e025f971485690864e4","eligible":true,"reason_code":"ok","issued_at":1786961117,"expires_at":1789553117,"digest":"74c0a6a5305227f29bff5a56a360f245d4bd3ba95c8603e9aba0f27f68cdf091","reused":false}

check-eligibility (second call, expect reused=true)
-> {"policy_id":"demo-adult-2","subject":"f39cbc5ab038a4b3fa862e025f971485690864e4","eligible":true,"reason_code":"ok","issued_at":1786961117,"expires_at":1789553117,"digest":"74c0a6a5305227f29bff5a56a360f245d4bd3ba95c8603e9aba0f27f68cdf091","reused":true}

verify-attestation (genuine)
-> {"valid":true,"reason_code":"ok","expires_at":1789553117}

verify-attestation (tampered digest, expect valid=false)
-> {"valid":false,"reason_code":"digest_mismatch","expires_at":1789553117}

reuse check: first.reused=false second.reused=true <- second call made no outbound request

[exit 0]
```

### Screenshot 3 — Proof the host resolves PII placeholders

```
$ POLICY_ID=demo-eu-only-2 npx tsx --env-file=.env scripts/demo-negative.ts

seeded policy 'demo-eu-only-2' allowing DE, FR (profile is NG)

check-eligibility('demo-eu-only-2')
-> {"policy_id":"demo-eu-only-2","subject":"f39cbc5ab038a4b3fa862e025f971485690864e4","eligible":false,"reason_code":"country_not_permitted","issued_at":1786961141,"expires_at":1789553141,"digest":"ae7c8be88062f75dcc32808f2f641eb60d17bcf4d39dd862614513fe9dd86a6","reused":false}

PROVEN: the verdict flipped on the profile's real residence_country.
The host resolved {{profile.residence_country}} to an actual value,
which the contract itself never saw.

[exit 0]
```

### Screenshot 4 — BUG-10 reproduced: tenant.me is not a function

```
$ npx tsx --env-file=.env scripts/demo-bug10.ts

Connected as: did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4

--- the documented call, verbatim from the docs ---
await tenant.me();

TypeError: tenant.me is not a function

--- where me() actually lives ---
TenantClient.prototype.me      : undefined
tenant.tenant.me               : function

the working call is: await tenant.tenant.me()
...which then fails with BUG-11 (SDK sends contract_id, node wants script_name)

[exit 0]
```

### Screenshot 5 — BUG-01 reproduced: the documented Rust snippet fails to compile

```

$ cd z-tenant-flight
# Apply the documentation instruction: "do not hex::encode it again"
sed -i "s|hex::encode(&tid)|tid|" src/search.rs
sed -n "180,183p" src/search.rs
echo
cargo build --target wasm32-wasip2 --release --offline 2>&1 | grep -A9 "^error\[| " | head -12
git checkout src/search.rs

fn get_api_key() -> Result<alloc::string::String, alloc::string::String> {
    let tid = tenant_context::tenant_id();
    let map_name = alloc::format!("z:{}:secrets", tid);
    let bytes = kv_store::get(&map_name, b"duffel_api_key")

error[E0277]: `Vec<u8>` doesn't implement `std::fmt::Display`
--> src/search.rs:182:51
   |
182 |         let map_name = alloc::format!("z:{}:secrets", tid);
   |                                     ^^^ `Vec<u8>` cannot be formatted with the default formatter
   |                                     required by this formatting parameter
   = help: the trait `std::fmt::Display` is not implemented for `Vec<u8>`
   = note: in format strings you may be able to use `{:?}` (or `{:#?}` for pretty-print) instead
Updated 1 path from the index

[exit 0]

```

## Screenshot 6 — 38 tests passing

```

$ echo "=== z-tenant-eligibility: Rust unit tests ==="
cd z-tenant-eligibility
cargo test --target x86_64-unknown-linux-gnu --offline 2>&1 | grep -E "^test [a-z]|test result" | head -26
cd ..
echo
echo "=== verifier: TypeScript tests ==="
npx tsx verifier/test.ts 2>&1 | tail -6

=== z-tenant-eligibility: Rust unit tests ===
test attest::tests::non_wasm_returns_err ... ok
test attest::tests::rejects_bad_json ... ok
test attest::tests::rejects_inline_pii_before_anything_else ... ok
test attest::tests::rejects_missing_policy_id ... ok
test attest::tests::rejects_policy_id_with_separator ... ok
test policy::tests::accepts_clean_input ... ok
test policy::tests::attestation_key_is_unambiguous ... ok
test policy::tests::attestation_carries_no_pii_fields ... ok
test policy::tests::check_req_rejects_non_json ... ok
test policy::tests::rejects_inline_pii ... ok
test policy::tests::digest_changes_when_verdict_changes ... ok
test policy::tests::digest_is_32_bytes_as_set_claims_digest_requires ... ok
test policy::tests::digest_is_stable_across_calls ... ok
test policy::tests::rejects_policy_id_that_would_collide_in_composite_key ... ok
test tests::contract_version_is_semver ... ok
test policy::tests::verifier_response_drops_unexpected_fields ... ok
test policy::tests::verifier_body_keeps_placeholders_literal ... ok
test verify::tests::constant_time_eq_matches_semantics_of_normal_eq ... ok
test verify::tests::rejects_bad_json ... ok
test verify::tests::non_wasm_returns_err ... ok
test verify::tests::rejects_non_hex_digest ... ok
test verify::tests::rejects_non_hex_subject ... ok
test policy::tests::rejects_policy_id_carrying_a_placeholder_marker ... ok
test result: ok. 23 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
test src/lib.rs - __export_tenant_eligibility_impl (line 65) ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.12s

=== verifier: TypeScript tests ===
ok empty allowed_countries permits any country
ok an unresolved_placeholder is incomplete, not a literal comparison
ok missing subject is incomplete
ok response never carries the input values back

15 passed

[exit 0]

```

## Summary

---

I completed the Quickstart and the full five-part TEE contract walkthrough, then went beyond it: I wrote and deployed an **original Rust TEE contract** that demonstrates a use case the reference showcase does not cover, backed by a real external verifier service I also built and deployed.

Along the way I found **16 bugs**, including three that mean **the published Quickstart cannot run as written**. One of them is the top unanswered question on this bounty's own listing.

Everything below is reproducible. The repo contains raw transcripts in `evidence/`, a minimal compiler-verified reproduction in `repro/`, and the node's own request ids for each server-side failure.

---

## Part 1 — The published Quickstart does not run

---

Three separate defects sit between a new developer and their first authenticated call.

### BUG-09 (Blocker) — `T3nClient` requires `trustAnchor`, which the Quickstart omits

The very first code sample on the "working in under 10 minutes" page throws:

```
T3nConfigError: T3nClient: `trustAnchor` is required and must be either a
TrustAnchor ({ expected_peer_ids, rtmr3_allowlist }) that pins the node's DKG
attestation, or the explicit opt-out { unsafe_trust_server: true }.
  code: 'CONFIG_ERROR', field: 'trustAnchor'
```

The SDK's own types say "Required (no silent default)", so the docs were written against an older SDK.

**Fix** — the secure path, not the opt-out:

```
import { fetchTrustedManifest, getEnvironment } from "@terminal3/t3n-sdk";

const t3n = new T3nClient({
  wasmComponent,
  trustAnchor: await fetchTrustedManifest(getEnvironment()),
  handlers: { EthSign: metamask_sign(address, undefined, apiKey) },
});
```

Please don't document `{ unsafe_trust_server: true }` as the quick fix — `fetchTrustedManifest`

verifies against an SDK-pinned operator key and is barely more code.

### BUG-10 (Blocker) — `tenant.me()` does not exist

**This is the top comment on this bounty's listing**, from a developer who never got an answer:

*"stuck at Set Up Development Environment, the `await tenant.me()` line keeps throwing an error and the troubleshooting table don't show a fix for it"*

The documented success check throws `TypeError: tenant.me is not a function. me() is not on TenantClient` — it lives on the tenant namespace:

```
$ node -e "import('@terminal3/t3n-sdk').then(m=>console.log(
  typeof m.TenantClient.prototype.me, typeof m.TenantNamespace.prototype.me))"
undefined function
```

So the call is `tenant.tenant.me()`. That still fails, because of:

### BUG-11 (Blocker) — `TenantClient`'s control plane is wire-incompatible with the node

```
RpcError: Invalid action request: missing field `script_name` at line 1 column 105
requestId: ec5aa3b0-6ae9-4352-85bf-22edd2bcfff9
```

`TenantClient.controlPayload` builds `{ contract_id, contract_version, ... }`, but the node's `action.execute` wants `script_name / script_version` — the same names the invoke walkthrough already uses for `T3nClient.execute`. The tenant path and the agent path disagree, and the tenant path is wrong.

Because every `tenant.*`, `maps.*` and `contracts.*` helper routes through `executeControl`, **the entire documented deployment path is unusable via the SDK.**

---

## Part 2 — How I got around it

### BUG-12 (Major) — unknown functions return `Internal` error, which masks everything

```
me()                -> -32603 Internal error [10d424b7-...]
zzz-not-a-real-function -> -32603 Internal error [b57b60c2-...]
```

versus a *real* function given bad arguments:

```
agent-auth-update -> -32602 role-assignments edge validation failed:
    grants[0] invalid: invalid contract_id: z:deadbeef:noop
```

A client-side name typo is indistinguishable from a broken node, and the docs tell you a generic 500 means "retry, then file a support ticket". That is why BUG-10's real cause went unexplained.

That asymmetry is also what made the rest of this possible: **-32602 means the function exists**, **-32603 means it doesn't**, so the node can be probed for its own API surface.

### BUG-13 (Blocker) — SDK and node disagree on names throughout

Recovered by probing `tee:tenant/contracts@1.26.0`:

```
absent      me
absent      claim
EXISTS (ok)  tenant-me
EXISTS (params) map-create      -> missing field `map_name`
EXISTS (params) map-entry-set    -> missing field `map_name`
EXISTS (params) contract-register -> missing field `name`
```

| Operation         | SDK sends    | Node wants                                             |
|-------------------|--------------|--------------------------------------------------------|
| tenant identity   | me           | tenant-me                                              |
| create map        | tail         | map_name, fully canonical                              |
| register contract | tail         | name, fully canonical                                  |
| register payload  | wasn't field | <b>multipart blob</b> via <code>executeWithBlob</code> |

Bare tails are rejected outright, which settles a documentation contradiction (BUG-06) in favour of the full-name form:

```
canonical map name invalid: must start with `z:f39cbc5ab038a4b3fa862e025f971485690864e4:`
```

The WASM is not a JSON field at all — `{ name, version }` alone gives `host:stash.persist-attached-blob: StashError::NoBlobAttached`.

### Working registration:

```
await t3n.executeWithBlob(
  { script_name: "tee:tenant/contracts", script_version: "1.26.0",
    function_name: "contract-register",
    input: { name: `z:${tid}:eligibility`, version: "0.1.5" } },
  new Blob([wasm], { type: "application/wasm" })),
);
// -> { name: "z:f39cbc...:eligibility", contract_id: 717 }
```

Once past this, `tenant-me` confirmed the account was fine all along:

```
{ "tenant": "did:t3n:f39cbc5a...", "label": "testnet-dev", "status": "active",
  "quotas": { "max_contracts": 10, "max_maps": 50, "max_wasm_bytes": 1048576, ... } }
```

---

## Part 3 — The Rust walkthrough

---

### BUG-01 (Blocker) — the docs contradict the reference implementation, and the snippet does not compile

The walkthrough says, in a comment and again under Key Design Rules, that the tenant DID is already a string and must **not** be hex-encoded:

```
// tenant_did() already returns the tid as a string – do not hex::encode it again.
// (Wrapping it in hex::encode a second time is a real bug some teams have hit...)
let tid = tenant_context::tenant_did();
let map_name = format!("z:{}.secrets", tid);
```

All three of these contradict it:

1. **The WIT declares bytes** — `tenant-did: func() -> list<u8>`; ("the 20-byte raw CompactDid shape"), in the repo the walkthrough tells you to clone.
2. **It does not compile.** Applying the documented instruction to Terminal 3's own reference contract:

```
error[E0277]: `Vec<u8>` doesn't implement `std::fmt::Display`
--> src/search.rs:182:51
```

3. **The reference implementation does the opposite** — `src/search.rs:182` and `src/booking.rs:171` both hex-encode.

So a developer following the docs faithfully gets a compile error, and the obvious fix is the exact thing the docs just called a known bug. The same wrong rule is repeated on the Common Errors page, so troubleshooting confirms the mistake instead of correcting it.

Reproduction, as a one-line patch plus captured `rustc` output: `repro/bug-01/` in the repo.

### BUG-02 (Major) — the documented `Cargo.toml` omits `hex`

The reference repo needs `hex = { version = "0.4", ... }` and uses it in two files; the doc's manifest lists only three dependencies. Copying it gives `E0433: failed to resolve: use of undeclared crate or module 'hex'` — a second error from the same under-tested sample.

### BUG-03 (Major) — `getScriptVersion` does not exist

The invoke walkthrough calls it twice. The real export is `getContractVersion`, same signature. A plain rename fixes it.

---

## Part 4 — Deployment and runtime findings

---

### BUG-15 (Major) — re-registering silently orphans every map ACL

Each `contract-register` mints a **new** `contract_id` (I observed `712 → 713 → 714 → 715 → 716 → 717` for one contract). Map ACLs are keyed by `contract_id`, and `map-create` is idempotent — it returns "already exists" without refreshing the ACL. So the second deploy breaks at runtime:

```
access denied: TenantContract(did:t3n:f39cbc.../714) cannot read map
"z:f39cbc...:attestations" (req f3dc2b01-d584-4c8b-a52c-c8fd88e217f3)
```

The fix is to call `map-update` on *every* deploy, not just the first.

### BUG-16 (Major) — the user-profile schema is undocumented, and the field is not `country`

Placeholder resolution is the headline feature, but nothing lists the valid `{{profile.<field>}}` names. `{{profile.country}}` fails only after a full build-register-invoke cycle:

```
contract error: eligibility verifier: user profile missing field: country
```

`user-upsert` rejects unknown keys and names them, which makes the schema recoverable:



```
REJECTED: citizenship, country, country_code, email, family_name, full_name,
          given_name, location, nationality, phone, region, residence
ACCEPTED: date_of_birth, first_name, last_name, residence_country, address,
          verified_contacts, gender
```

The field is `residence_country`. Note the trap: `country` is both the obvious name and an explicitly rejected key.

### **BUG-08 (Blocker) — the three-principal invoke flow can't be completed with a claimed credential**

Since this bounty is titled "Create Agent ID", I tested the agent path properly rather than relying on the self-call fallback. Step 4 of the walkthrough needs `AGENT_KEY` and `USER_KEY` beyond the tenant key, tells you not to reuse the tenant key, and never says where they come from. So I generated an agent key locally and traced exactly how far it gets.

**A self-generated key does yield a valid, distinct DID, and authenticates:**

```
tenant/user : did:t3n:f39cbc5ab038a4b3fa862e025f971485690864e4
agent       : did:t3n:15e4b33b0886f32b63dc9f9a614ad8db17a6e463
same DID?   : no – genuinely distinct
```

**The delegation grant succeeds too** — the data owner scopes the agent to the contract, its functions and its egress hosts, and the node accepts it:

```
granted did:t3n:15e4b33b0886f32b63dc9f9a614ad8db17a6e463
-> z:f39cbc...:eligibility (hosts: verifier-jade.vercel.app)
```

**But the agent cannot execute anything:**

```
[403] InsufficientCredit (account=15e4b33b..., required=100000000000, available=0)
      (req 9be2381f-ddbf-4c9d-a247-69d6605d253d)
```

At `TOKEN_DECIMALS = 6` that is **10,000 tokens for a single invocation**, against zero. It cannot bootstrap itself either:

```
[-32602] email_not_verified: caller has no verified email and supplied no
proving authenticator. Run otp-request + otp-verify first.
      (req bfbcd318-4b38-477a-b2b1-fd5f73747bee)
```

And there's no self-service funding path — nine plausible token function names on `tee:tenant/contracts` (`token-balance`, `token-transfer`, `token-grant`, `tenant-usage`, ...) all return absent.

The sting is that the **grant succeeding** signals the setup is right; the failure lands one call later, where the natural suspicion is the contract rather than the identity. Leading step 4 with the self-call form, and having `agent-auth-update` warn when granting to an unfunded DID, would fix this cheaply.

## Remaining findings

| #          | Severity       | Summary                                                                                                                                                                             |
|------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BUG-0<br>6 | Major          | Two pages disagree on bare tail vs full <code>z:&lt;tid&gt;:</code> map name (the node settles it: full name)                                                                       |
| BUG-0<br>8 | <b>Blocker</b> | The three-principal invoke flow cannot be completed with a claimed credential — see below                                                                                           |
| BUG-0<br>7 | Minor          | The tenant world's available host interfaces are never listed                                                                                                                       |
| BUG-1<br>4 | Minor          | Two different credential formats are both called "the API key" ( <code>0x...</code> signing key vs <code>t3n_key_...</code> )                                                       |
| BUG-0<br>4 | Minor          | <code>idx:_tenants</code> admission is referenced but never explained — I initially thought this was the blocker; testing proved it is automatic, and I've recorded that correction |
| BUG-0<br>5 | Minor          | <code>npm install EBADENGINE</code> warning, no documented Node range (it is benign; the SDK works on Node 20)                                                                      |

---

## Part 5 — Going beyond: an original TEE contract

---

The reference contract shows PII-safe *booking*. I wanted to show something harder to build anywhere else: **answering a question about a user's PII and keeping only the answer**.

The problem is everywhere. To gate anything on "is this person over 18 and in a permitted country?", the usual approach collects a date of birth and an address — so every tenant that needs the answer ends up storing regulated data it never wanted, and re-collects it on every check.

`z-tenant-eligibility` inverts that:

- **The contract never sees the PII.** The predicate is evaluated by an external verifier reached via `http-with-placeholders`. The body carries `{{profile.date_of_birth}}` and `{{profile.residence_country}}` markers, still literal text inside the WASM; the host substitutes them on its own stack after the contract has composed the request.
- **The caller never sees it either.** Only `{ eligible, reason_code, issued_at, expires_at }` crosses the WIT boundary. Reason codes are coarse by design, since a precise reason leaks a bit of the profile.
- **The answer is reusable.** Cached per policy per user — a second call inside the window returns `reused: true` and makes **no outbound call at all**, so the profile is read once per window rather than once per check.
- **The subject cannot be forged.** Taken from `tenant-context.calling-user-did()`, never from contract input.
- **It is tamper-evident.** `kv-store.set-claims-digest` binds the attestation's SHA-256 into the transaction's Merkle leaf, so a holder can prove from the ledger receipt that this cluster issued exactly this record.

## It works, live on testnet

```
// first call – resolves PII host-side, calls the verifier, issues an attestation
{"policy_id":"adult-eu","subject":"f39cbc5a...","eligible":true,"reason_code":"ok",
 "digest":"51fc6b1b5d75923e...","reused":false}

// second call – same digest, no outbound request, profile not read
{... "digest":"51fc6b1b5d75923e...","reused":true}

// verify genuine, then tampered
{"valid":true,"reason_code":"ok","expires_at":1789550459}
{"valid":false,"reason_code":"digest_mismatch","expires_at":1789550459}
```

## Proving the PII really is resolved host-side

A positive result alone proves little — a verifier ignoring its input would also say "eligible". So the verifier returns `profile_incomplete` if it ever sees a literal `{{` marker, and I seeded a second policy

whose `allowed_countries` **excludes** the profile's real country:

```
seeded policy 'eu-only' allowing DE, FR (profile is NG)
-> {"policy_id":"eu-only","eligible":false,"reason_code":"country_not_permitted", ...}
```

The verdict flipped on a value the contract never held. The three outcomes are distinguishable — `ok`, `country_not_permitted`, `profile_incomplete` — and we got the one that proves resolution happened.

## The verifier

Deployed at <https://verifier-jade.vercel.app/api/eligibility>. It is deliberately the only component that ever holds plaintext PII, and only for one request: it never logs a resolved value, never echoes one back, and persists nothing. Age is computed calendar-correctly rather than by dividing days by 365.25, because a minimum-age check is a cliff and the naive version is wrong near a birthday.

## Where it could go

The natural next step is `signing::sign`, turning the attestation into a portable credential a *different* tenant could verify without querying this one — the reusable-verified-data story T3N is built for. I didn't, only because the docs never state which host interfaces a tenant world provides (BUG-07), so importing it is an untestable guess.

---

## Testing

38 tests, none requiring a live host:

- **23 Rust** — digest stability, PII-absence assertions on the returned attestation, inline-PII rejection, KV key ambiguity, constant-time digest comparison, and that `{{profile.*}}` markers survive serialization unresolved.
- **15 TypeScript** — verifier logic, with the age boundary tested on the exact birthday and the day either side.

```
(cd z-tenant-eligibility && cargo test --target x86_64-unknown-linux-gnu)
npx tsx verifier/test.ts
```

---

## Environment

---

|                      |                                    |
|----------------------|------------------------------------|
| OS                   | Parrot / Debian 13, Linux 6.17.13  |
| Node                 | v20.19.2, npm 9.2.0                |
| Rust                 | rustc 1.97.1, target wasm32-wasip2 |
| SDK                  | @terminal3/t3n-sdk@4.39.1          |
| wasm-tools           | 1.256.0                            |
| Cluster              | cn-api.sg.testnet.t3n.terminal3.io |
| tee:tenant/contracts | 1.26.0                             |
| tee:user/contracts   | 2.25.1                             |

---

## One open question I could not answer

---

Recorded rather than glossed over, because it affects my own contract.

**What does `calling-user-did()` return during a delegated call?** The WIT calls it "the authenticated session DID". When an *agent* invokes on a user's behalf, either reading is plausible:

- it returns the **user's** DID → `z-tenant-eligibility` is correct, since the attestation is attributed to whoever's PII was read; or
- it returns the **agent's** DID → my contract has a genuine correctness bug, because the host resolves the *user's* profile for the `{{profile.*}}` placeholders while the attestation would be keyed to the agent. One person's eligibility would be recorded against a different principal.

I wrote the test (`scripts/agent-test.ts`, step 5) and it's ready to run, but it cannot reach that step — the agent has no credits (BUG-08), so the call never executes. Everything in this report used the self-call form, where user and caller are the same DID and the ambiguity doesn't arise.

Flagging it because any contract keying authorisation or record-ownership on `calling-user-did()`

behaves differently under delegation depending on which reading holds, and the docs don't say. If you can fund an agent identity on your side, that test answers it in one call — I'd genuinely like to know.

## What would help most

---

If I could change one thing, it would be **BUG-12** — returning a proper `-32601 Method not found` instead of `Internal error`. Every other defect here took hours longer than it should have because a name mismatch and a genuine server fault look identical, and the documented response to both is "file a ticket". Fixing that one makes the platform self-describing, which is how I eventually got everything working.

The second would be a round-trip integration test between the SDK's tenant helpers and a live node. BUG-10, BUG-11 and BUG-13 are all mechanical drift that a single such test would catch.